Université du Luxembourg

Introduction to Message Passing Interface (MPI)

Lesson 5

Matteo Barborini

University of Luxembourg - HPC Platform

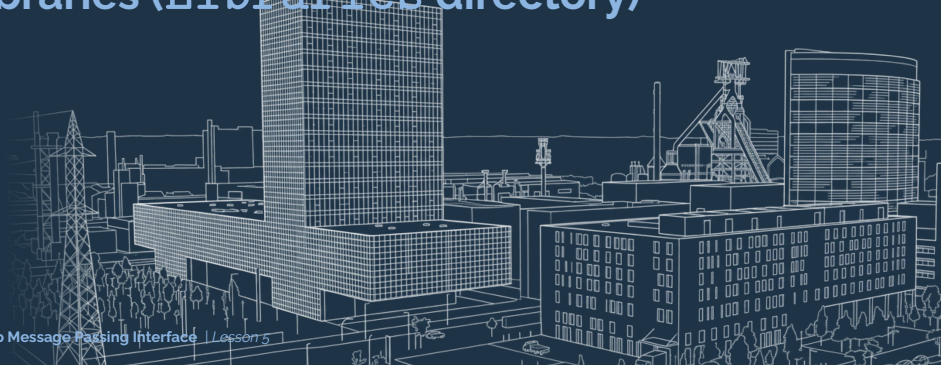
matteo.barborini@uni.lu

14th of October 2026



Université du Luxembourg

Usefull libraries (`Libraries` directory)



arrays.h

Usefull libraries (`Libraries` directory)

This library contains some useful function in C to allocate and initialize arrays and matrices.

In particular, the solutions `SERIAL_dgemv.c` and `MPI_dgemv.c` use some of these functions.

```c
// Allocate empty matrix
double **allocate_2d_double(int *n, int *m);

// Allocate packed empty matrix
double **allocate_2d_packed_double(int *n, int *m);

// Allocate empty vector
double *allocate_1d_double(int *n);

// Print matrix
void print_2d_double(int *n, int *m, double **A);

// Print vector
void print_1d_double(int *n, double *a);

// Initialize a random matrix  with elements from [-1:1]
void init_random_matrix(int seed,int *n, int *m, double **A);

// Initialize a random vector  with elements from [-1:1]
void init_random_vector(int seed, int *n, double *a);
```

Matrix allocations in C

Usefull libraries (`Libraries` directory)

Examples of matrix allocations in C can be found in the example `4.1.MatrixAllocations.c`

The simplest way to allocate matrices in C is to first allocate a vector of pointers and then for each pointer a vector of length column.

```c
// Allocate empty matrix
double **allocate_2d_double(int rows, int columns)
{
    if (rows <= 0 || columns <= 0)
        return NULL;

    // Allocate memory for the row pointers (an array of double*)
    double **matrix = (double **)malloc(rows * sizeof(double *));

    // Allocate memory for the elements of each row (and initialize to 0)
    for (int i = 0; i < rows; i++)
    {
        matrix[i] = (double *)calloc(columns, sizeof(double));
    }
    return matrix;
}
```

The memory is not necessarily contiguous!! What can happen in MPI transfers of non-contiguous data?

Matrix allocations in C

Usefull libraries (`Libraries` directory)

An alternative way to allocate contiguous memory for a matrix is to allocate a single vector of length rows*columns.

If we define the function

```c
// Allocate empty vector
double *allocate_1d_double(int elements)
{
    if (elements <= 0)
        return NULL;

    // Allocate memory for the pointers (an array of double*)
    double *vector = (double *)calloc(elements, sizeof(double));
    return vector;
}
```

We simply allocate a vector of length rows*columns

```c
vector = allocate_1d_double(rows * columns);
```

The element i, j of the matrix can then be accessed as `vector[i*columns + j]` assuming the allocation is again "columns first".

Remember that in Fortran it is the opposite, Fortran uses "rows first" allocation.

Matrix allocations in C

Usefull libraries (`Libraries` directory)

An alternative way to allocate contiguous memory for a matrix is to allocate a single temporary vector of length `rows * columns`.

Afterwards, the matrix can be allocated and the pointers can be associated to the ones used for the larger vector.

```c
// Allocate empty matrix (consecutive elements)
double **allocate_2d_double_blocked(int rows, int columns)
{
    if (rows <= 0 || columns <= 0)
        return NULL;

    /* allocate the n*m contiguous items */
    double *temp_matrix = (double *)malloc(rows * columns * sizeof(double));

    /* allocate the row pointers into the memory */
    double **matrix = (double **)malloc(rows * sizeof(double *));

    /* set up the pointers into the contiguous memory */
    for (int i = 0; i < rows; i++)
        matrix[i] = &(temp_matrix[i * columns]);

    return matrix;
}
```

In this way the memory is contiguous and we can use the matrix as a 2D array.

linearalgebra.h

Usefull libraries (`Libraries` directory)

The linear algebra library `linearalgebra.h` contains the functions used in the previous exercises, in particular the dot product

```
double ddot(int *n, double *a, double *b)
{
   double result;
   result = 0.0;
   for (int i = 0; i < *n; i++)
   {
      result += a[i] * b[i];
   }
   return result;
}
```

and the matrix vector

```
void dgemv(int *n, int *m, double **A, double *b, double *c)
{
   for (int i = 0; i < *n; i++)
   {
      c[i] = 0.0;
      for (int j = 0; j < *m; j++)
      {
         c[i] += A[i][j] * b[j];
      }
   }
}
```

linearalgebra.h

Usefull libraries (`Libraries` directory)

The name used in the library are inspired by the classification used in the BLAS library.

For example :

(d)(dot) : **(d)** Data Precision (double) and Type; **(dot)** Level 1 (Vector-Vector Operations)

(d)(ge)(mv) : **(d)** Data Precision (double) and Type; **(ge)** Matrix Structure / Data Type (Letters 2 and 3); **(mv)** Type or operation in this case matrix-vector.

So for matrix-matrix multiplication of double precision numbers we would have

(d)(ge)(mv) : **(d)** Data Precision (double) and Type; **(ge)** Matrix Structure / Data Type (Letters 2 and 3); **(mm)** Type or operation in this case matrix-matrix.

mt19937ar.h

Usefull libraries (`Libraries` directory)

The library `mt19937ar.h` contains the Mersenne Twister pseudo-random number generator that has a period of $2^{19937} - 1$[1] (This will also be usefull for future exercises) .

The library is used in the `arrays.h` library to initialize the matrices and vectors with random numbers in the interval [-1:1].

In general, to initialize the random number generator (**always define a seed to guarantee reproducibility**).

To initialize the random number generator call

```
1 init_genrand(seed);
```

where `seed` is an integer number.
Calling

```
1 double random_number=genrand_real1();
```

The `random_number` variable is initialized with a random double precision number in the interval [0:1].

[1] M. Matsumoto and T. Nishimura, "Mersenne twister: a 623-dimensionally equidistributed uniform pseudo-random number generator", ACM Trans. Model. Comput. Simul. **8**, 3–30 (1998).

Université du Luxembourg

Exercise: Matrix-Vector multiplication and Euclidean Norm



Definition of matrix-vector multiplication and the norm of a vector

Exercise: Matrix-Vector multiplication and Euclidean Norm

We consider a matrix $\mathbf{A}$ with *N* rows and *M* columns, and a vector $\mathbf{b}$ of length *M*, both containing double precision elements.

We define the **matrix-vector product** between the transposed of $\mathbf{A}$ and the vector $\mathbf{b}$ as $\mathbf{c} = \mathbf{A}\mathbf{b}$ and the single element of the $\mathbf{c}$ will be given by

$$\mathbf{c}_i = \sum_{j=1}^{N} \mathbf{A}_{ij}\mathbf{b}_j.$$

We define the **dot product** between two vectors as

$$d = \mathbf{a} \cdot \mathbf{b} = \sum_{i=1}^{N} a_i b_i, \tag{1}$$

and **the norm** $\mathcal{N}$ of a vector as:

$$\mathcal{N} = \sqrt{\mathbf{a} \cdot \mathbf{a}} = \sqrt{\sum_{i=1}^{N} a_i^2}. \tag{2}$$

Exercise: Compute the norm of a vector $\mathbf{c}$ result of $\mathbf{c} = \mathbf{Ab}$

Exercise: Matrix-Vector multiplication and Euclidean Norm

Exercise: Write an MPI code to parallelize the computation of the norm $\mathcal{N}$ of a vector *c*, result of a Matrix-Vector operation do a Matrix vector operation $\mathbf{c} = \mathbf{Ab}$.
The code will use any type of functions that you prefer, in particular `MPI_*Send/MPI_*Recv`, `MPI_*Bcast` and `MPI_*Reduce`.
Measure the times for "Initialization", "Matrix operations", "Reduce Operations" with the MPI function `MPI_Wtime()`.

1. Rank 0 Initializes matrix $\mathbf{A}$ (128000×1000 elements)and vector $\mathbf{b}$ (1000 elements) with random double precision numbers distributed in the interval [-1:1] using the functions in `array.h`. The initial random seed to be used is 10.
2. Rank 0 communicates to all other ranks the vector $\mathbf{b}$ and parts of the matrix $\mathbf{A}$.
3. Each process executes the local matrix-vector product using the functions in `linearalgebra.h`, obtaining parts of the of the vector $\mathbf{c}$.
4. Each process computes parts of the dot product of the vector $\mathbf{c}$ with itself (parts of the $\sum_{i=1}^{N} c_i^2$ sum) necessary to compute the norm (`ddot` in `linearalgebra.h`).
5. The results of the partial dot products obtained for each process are summed through a reduce operation on rank 0.
6. Rank 0 computes the square root of the dot product $\sqrt{\sum_{i=1}^{N} c_i^2}$, equal to the norm of the vector $\mathbf{c}$
7. Rank 0 prints the value of the norm.

Speed Up in dgemv with MPI

Exercise: Matrix-Vector multiplication and Euclidean Norm

